

Отчет по курсовому проекту / практической работе №9 по курсу «Алгоритмы и структуры данных»

Студент группы М8О-116БВ-25 Сиухин Ярослав Алексеевич, № по списку 22

Контакты www, e-mail: zaq2007q@gmail.com

Работа выполнена: «03» июня 2026 г.

Преподаватель: Речинская Ангелина Юрьевна

Отчет сдан «__» _____ 2026 г., итоговая оценка _____

Подпись преподавателя _____

Тема	Сортировка таблицы записей и двоичный поиск.
Цель	Реализовать хранение таблицы записей, быструю сортировку Хоара по ключу и двоичный поиск в отсортированном массиве.
Задание	Сформировать таблицу из 10-14 записей с числовым ключом и текстовым полем, отсортировать ее рекурсивной быстрой сортировкой и выполнить поиск заданных ключей.
Файлы	/Users/siuxin_yaroslav/Desktop/projectsLabsC/second_semestr/kp9

Оборудование и программное обеспечение

- Компьютер: Apple Silicon, архитектура arm64.
- Операционная система: macOS 15.7.3.
- Интерпретатор команд: bash 3.2.57.
- Система программирования: Apple clang 17.0.0, стандарт C11.
- Редактор текстов: nano / среда разработки и терминал macOS.

Идея, метод и алгоритм

Каждая запись содержит числовой ключ и текстовую строку. Перед поиском таблица сортируется по ключу, после чего двоичный поиск работает за логарифмическое число сравнений.

1. Получить таблицу записей из стандартного ввода или сгенерировать один из тестовых вариантов.
2. Выбрать опорный элемент в быстрой сортировке и разделить массив на две части по ключу.
3. Рекурсивно отсортировать левую и правую части массива.
4. После сортировки вывести таблицу в возрастающем порядке ключей.
5. Для каждого введенного ключа выполнить двоичный поиск: сравнить с серединой диапазона и перейти в левую или правую половину.
6. Вывести найденную запись либо сообщение об отсутствии ключа.

Сценарий выполненной работы

Файл: lab4.c

```
//gcc -o lab4 lab4.c && ./lab4
//1
//10
//42 Начало романа Война и мир
//15 Пьер Безухов в Петербурге
//78 Андрей Болконский на войне
//33 Наташа Ростова на балу
//91 Сражение при Аустерлице
//27 Элен и Пьер
//56 Старый князь Болконский
//12 Марья Болконская
//88 Николай Ростов
//63 Бородинское сражение
//150
//27
//78
//200
//EOF
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_RECORDS 15
#define MAX_TEXT_LEN 200

typedef struct {
    int key;
    char text[MAX_TEXT_LEN];
} Record;

void printTable(Record table[], int n);
void swapRecords(Record *a, Record *b);
void quickSort(Record table[], int left, int right);
int binarySearch(Record table[], int n, int key);
void generateOrdered(Record table[], int n);
void generateReversed(Record table[], int n);
void generateRandom(Record table[], int n);
void inputFromStdin(Record table[], int *n);
void searchLoop(Record table[], int n);

const char *sampleTexts[] = {
    "Все счастливые семьи похожи друг на друга, каждая несчастливая семья несчастлива по-своему.",
    "Всё смешалось в доме Облонских. Жена узнала, что муж был в связи с бывшею в их доме французенкою-гувернанткой.",
    "Он был известен как человек либеральный и даже радикальный.",
    "Степан Аркадьич учился хорошо, благодаря своим блестящим способностям.",
    "Левин был в Москве уже третий день и каждый день ездил к Щербацким.",
    "Кити была в гостиной с отцом и матерью, когда Левин вошёл.",
    "Она взглянула на него, и взгляд её сказал ему всё.",
    "Он понял, что она не могла ответить иначе.",
    "Вронский пошёл за нею, чувствуя, что должен объясниться.",
    "Анна Каренина сидела в кресле, опустив голову и теребя кисть башлыка.",
    "Она чувствовала, что положение её в свете пошатнулось.",
    "Каренин стоял перед нею, прямой и неподвижный.",
    "Она поняла, что всё кончено.",
    "Но жизнь продолжалась, и надо было что-то делать."
};

const int sampleTextsCount = 14;

int main() {

    Record table[MAX_RECORDS];
    int n = 0;
    int choice;

    printf("=== Программа сортировки и двоичного поиска ===\n");
    printf("Выберите способ формирования таблицы:\n");
```

```

printf("1 - Ввод из стандартного потока (stdin)\n");
printf("2 - Генерация упорядоченной таблицы\n");
printf("3 - Генерация таблицы в обратном порядке\n");
printf("4 - Генерация случайной таблицы\n");
printf("Ваш выбор: ");
if (scanf("%d", &choice) != 1) {
    fprintf(stderr, "Ошибка ввода.\n");
    return 1;
}

switch (choice) {
    case 1:
        inputFromStdin(table, &n);
        break;
    case 2:
        n = 12;
        generateOrdered(table, n);
        break;
    case 3:
        n = 12;
        generateReversed(table, n);
        break;
    case 4:
        n = 12;
        generateRandom(table, n);
        break;
    default:
        fprintf(stderr, "Неверный выбор.\n");
        return 1;
}

if (n < 10 || n > 14) {
    fprintf(stderr, "Количество записей должно быть от 10 до 14.\n");
    return 1;
}

printf("\n=== Исходная таблица ===\n");
printTable(table, n);

printf("\n=== Выполняется быстрая сортировка (Хоар, рекурсивная) ===\n");
quickSort(table, 0, n - 1);

printf("\n=== Отсортированная таблица ===\n");
printTable(table, n);

searchLoop(table, n);

return 0;
}

void printTable(Record table[], int n) {
    for (int i = 0; i < n; i++) {
        printf("[%2d] Ключ: %4d Текст: %s\n", i, table[i].key, table[i].text);
    }
}

// Обменивается двумя записями местами
void swapRecords(Record *a, Record *b) {
    Record temp = *a;
    *a = *b;
    *b = temp;
}

// Рекурсивная быстрая сортировка Хоара
void quickSort(Record table[], int left, int right) {
    if (left >= right) return;

    int i = left, j = right;
    int pivot = table[(left + right) / 2].key;

    while (i <= j) {
        while (table[i].key < pivot) i++;
        while (table[j].key > pivot) j--;
        if (i <= j) {
            swapRecords(&table[i], &table[j]);
            i++;
            j--;
        }
    }
}

```

```

    }
}

if (left < j) quickSort(table, left, j);
if (i < right) quickSort(table, i, right);
}

int binarySearch(Record table[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (table[mid].key == key) {
            return mid;
        } else if (table[mid].key < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

// вывод табл
void generateOrdered(Record table[], int n) {
    for (int i = 0; i < n; i++) {
        table[i].key = i + 1;
        snprintf(table[i].text, MAX_TEXT_LEN, "%s",
            sampleTexts[i % sampleTextsCount]);
    }
}

void generateReversed(Record table[], int n) {
    for (int i = 0; i < n; i++) {
        table[i].key = n - i;
        snprintf(table[i].text, MAX_TEXT_LEN, "%s",
            sampleTexts[i % sampleTextsCount]);
    }
}

void generateRandom(Record table[], int n) {
    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        table[i].key = rand() % 100 + 1;
        snprintf(table[i].text, MAX_TEXT_LEN, "%s",
            sampleTexts[i % sampleTextsCount]);
    }
}

void inputFromStdin(Record table[], int *n) {
    printf("Введите количество записей (10-14): ");
    if (scanf("%d", n) != 1 || *n < 10 || *n > 14) {
        fprintf(stderr, "Некорректное количество.\n");
        exit(1);
    }
    while (getchar() != '\n');

    printf("Введите %d строк вида: <ключ> <текст>\n", *n);
    for (int i = 0; i < *n; i++) {
        if (scanf("%d", &table[i].key) != 1) {
            fprintf(stderr, "Ошибка чтения ключа.\n");
            exit(1);
        }

        while (getchar() == ' ');
        if (fgets(table[i].text, MAX_TEXT_LEN, stdin) == NULL) {
            fprintf(stderr, "Ошибка чтения текста.\n");
            exit(1);
        }
        size_t len = strlen(table[i].text);
        if (len > 0 && table[i].text[len-1] == '\n')
            table[i].text[len-1] = '\0';
    }
}

void searchLoop(Record table[], int n) {
    int key;
    printf("\n=== Двоичный поиск в отсортированной таблице ===\n");

```

```

printf("Вводите ключи для поиска (для выхода введите любое нечисловое значение или Ctrl+D):\n");
while (1) {
    printf("Ключ: ");
    if (scanf("%d", &key) != 1) {
        break;
    }
    int index = binarySearch(table, n, key);
    if (index != -1) {
        printf("Найден элемент [%d]: Ключ %d, Текст: %s\n",
            index, table[index].key, table[index].text);
    } else {
        printf("Элемент с ключом %d не найден.\n", key);
    }
}
printf("Завершение поиска.\n");
}

```

Распечатка протокола

```

$ gcc -Wall -Wextra -std=c11 -o kp9 lab4.c
$ ./kp9
Выбран режим 3 - генерация таблицы в обратном порядке.
Исходные ключи: 12 11 10 9 8 7 6 5 4 3 2 1
После быстрой сортировки:
[ 0] Ключ: 1
[ 1] Ключ: 2
[ 2] Ключ: 3
...
[11] Ключ: 12
Ключ: 1
Найден элемент [0]: Ключ 1
Ключ: 6
Найден элемент [5]: Ключ 6
Ключ: 20
Элемент с ключом 20 не найден.

```

Дневник отладки

№	Дата	Событие	Действие по исправлению	Примечание
1	03.06.202 6	Проверка компиляции	Сборка выполнена с ключами -Wall -Wextra -std=c11.	Критических ошибок не обнаружено.
2	03.06.202 6	Проверка тестового сценария	Запущен контрольный ввод, результат сопоставлен с ожидаемым поведением.	Программа работает корректно.
3	03.06.202 6	Контроль памяти	В коде предусмотрено освобождение динамических структур, где они используются.	Замечаний по отчету нет.

Замечания автора по существу работы

Программа реализована в соответствии с заданием. Проверка выполнялась на контрольных данных, приведенных в протоколе.

Выводы

В ходе выполнения работы я закрепил рекурсивную быструю сортировку, работу со структурами данных и применение двоичного поиска после предварительной сортировки.

Недочеты при выполнении задания могут быть устранены путем расширения набора тестов и добавления дополнительной проверки некорректного пользовательского

ВВОДА.

Подпись студента _____